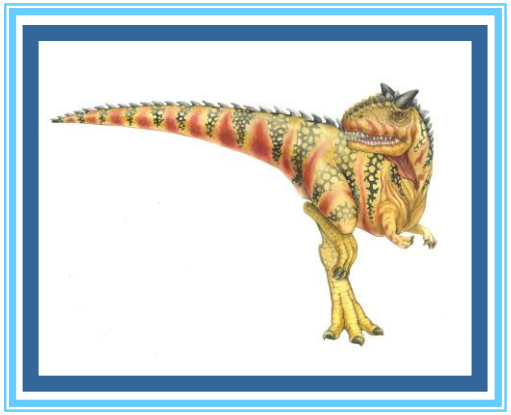
Chapter 4: Threads & Concurrency

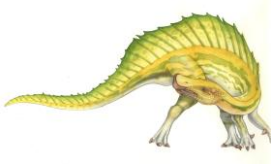


Silberschatz, Galvin and Gagne ©2018



Chapter 4: Threads

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues
- Operating System Examples





Objectives

- Identify the basic components of a thread, and contrast threads and processes
- Describe the benefits and challenges of designng multithreaded applications
- Illustrate different approaches to implicit threading including thread pools, fork-join, and Grand Central Dispatch
- Describe how the Windows and Linux operating systems represent threads
- Design multithreaded applications using the Pthreads, Java, and Windows threading APIs





Motivation

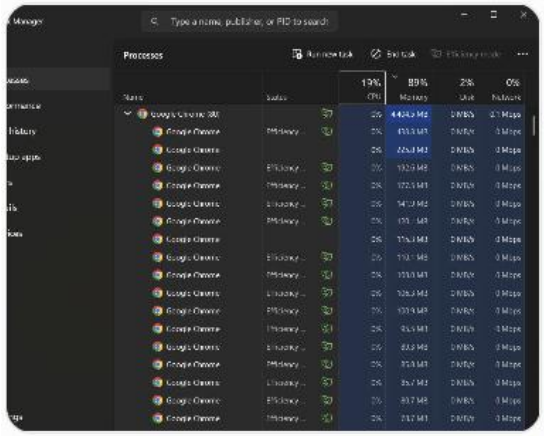
- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

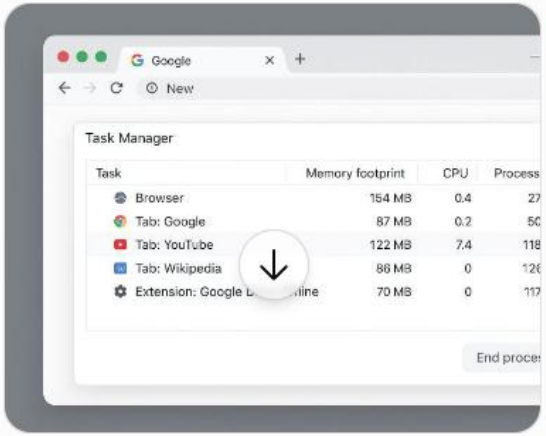


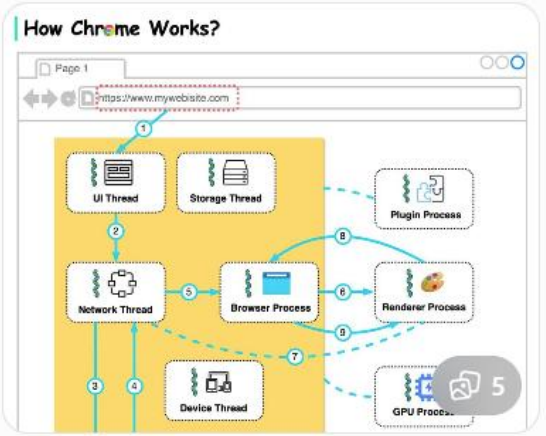


Examples

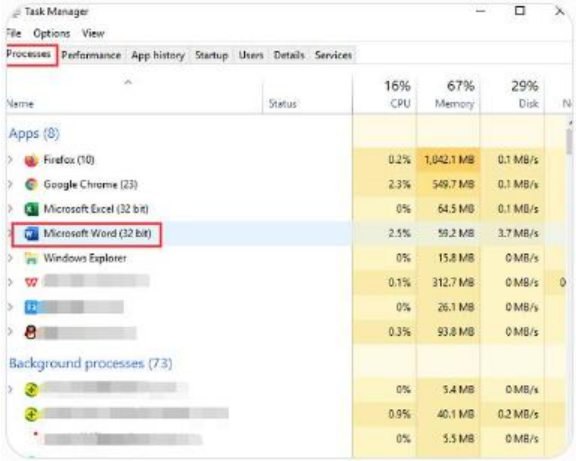
Google Chrome

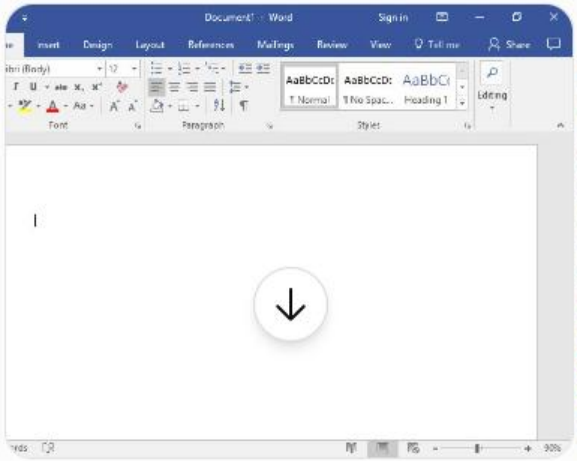


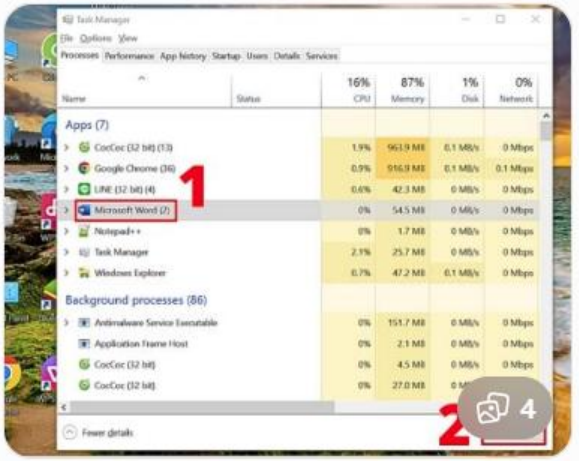




Microsoft word



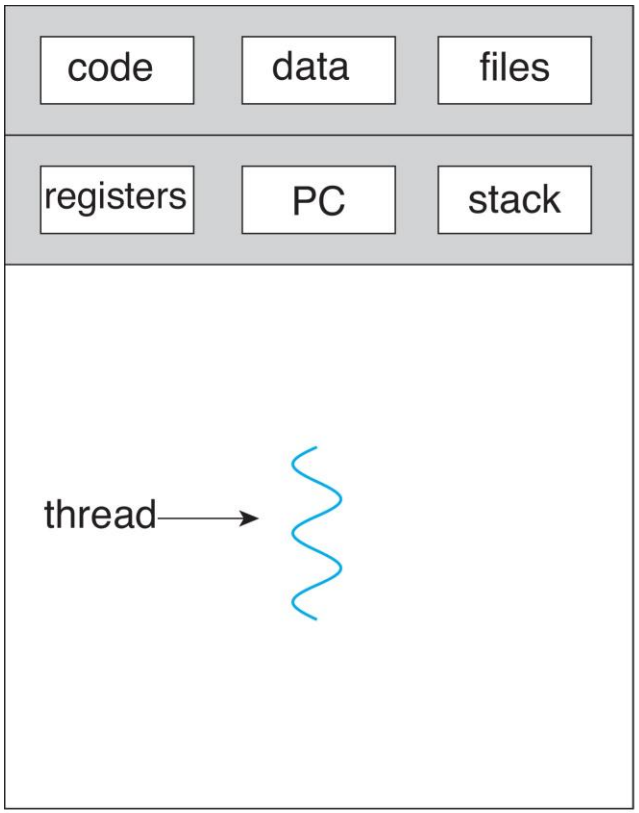




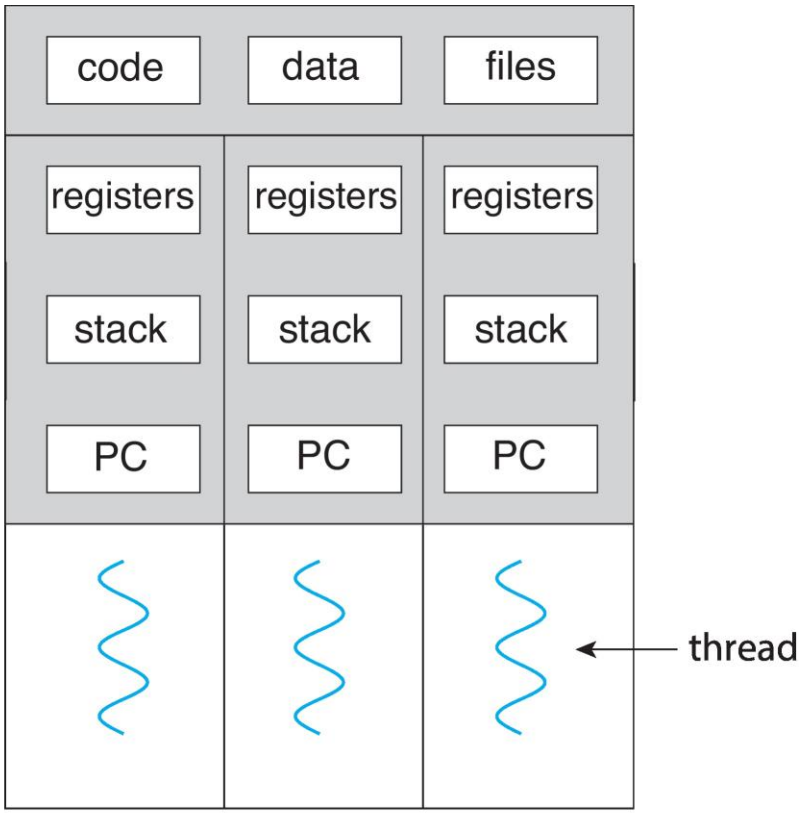




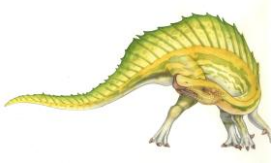
Single and Multithreaded Processes



single-threaded process

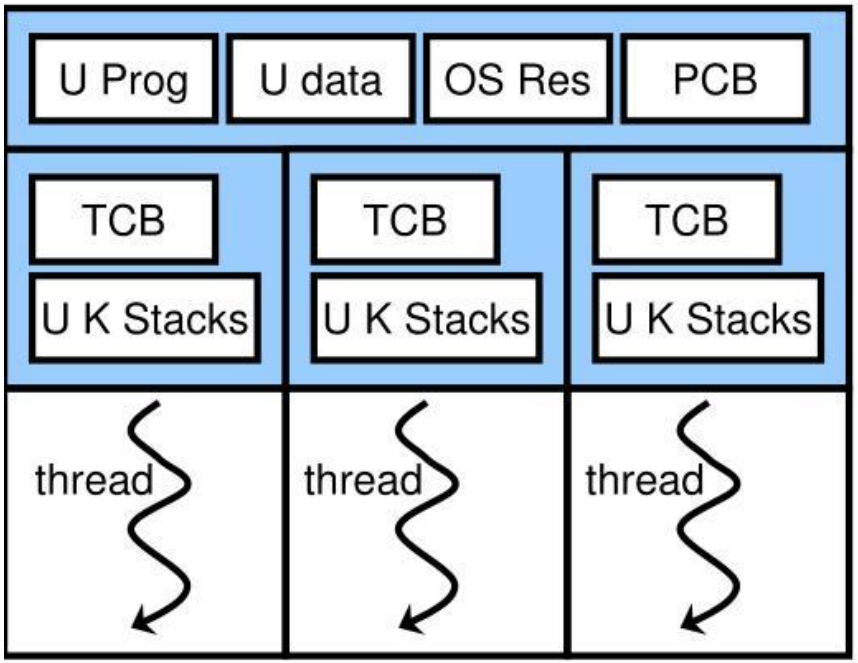


multithreaded process





Single and Multithreaded Processes

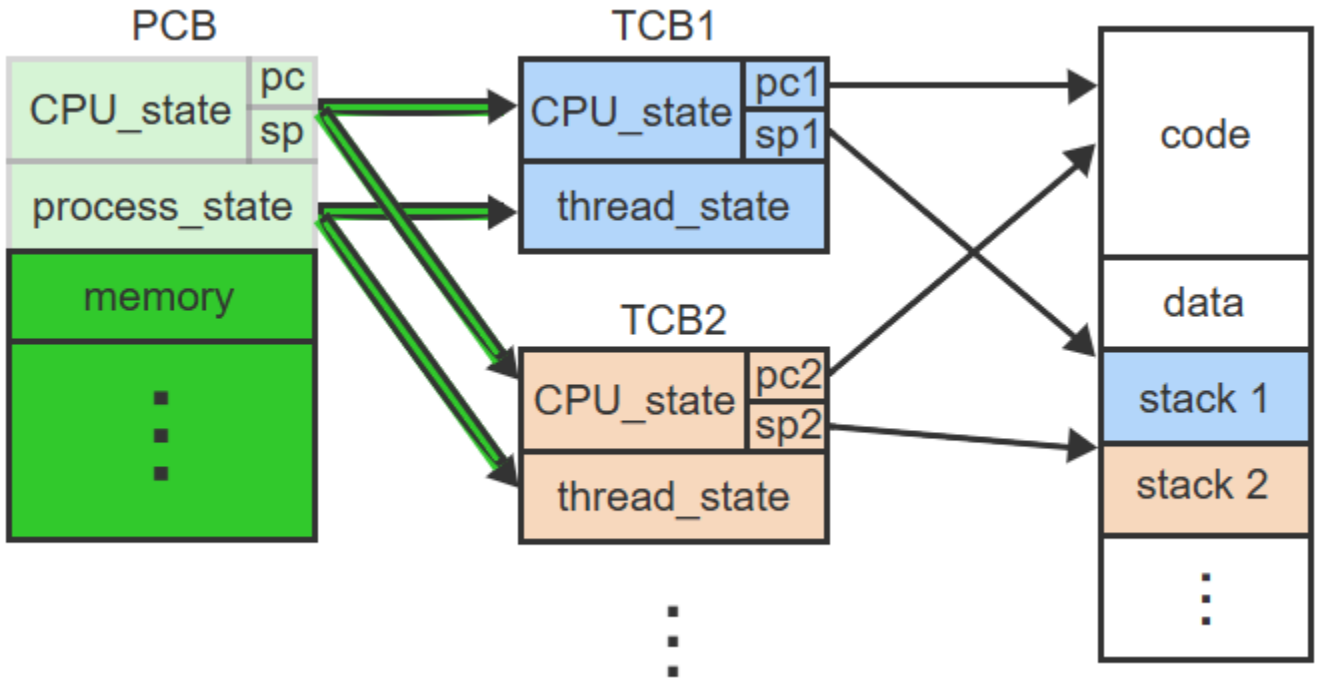


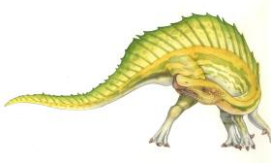
- Thread Control Block (TCB)
  - Thread ID
  - The contents of CPU registers
  - Program counter
    - Indicates the address of the next instruction to be executed.
  - Event that a thread is waiting on.
  - Scheduling info
- Different threads can share the memory and the resources used by a process.
  - User program
  - User data
  - OS resources





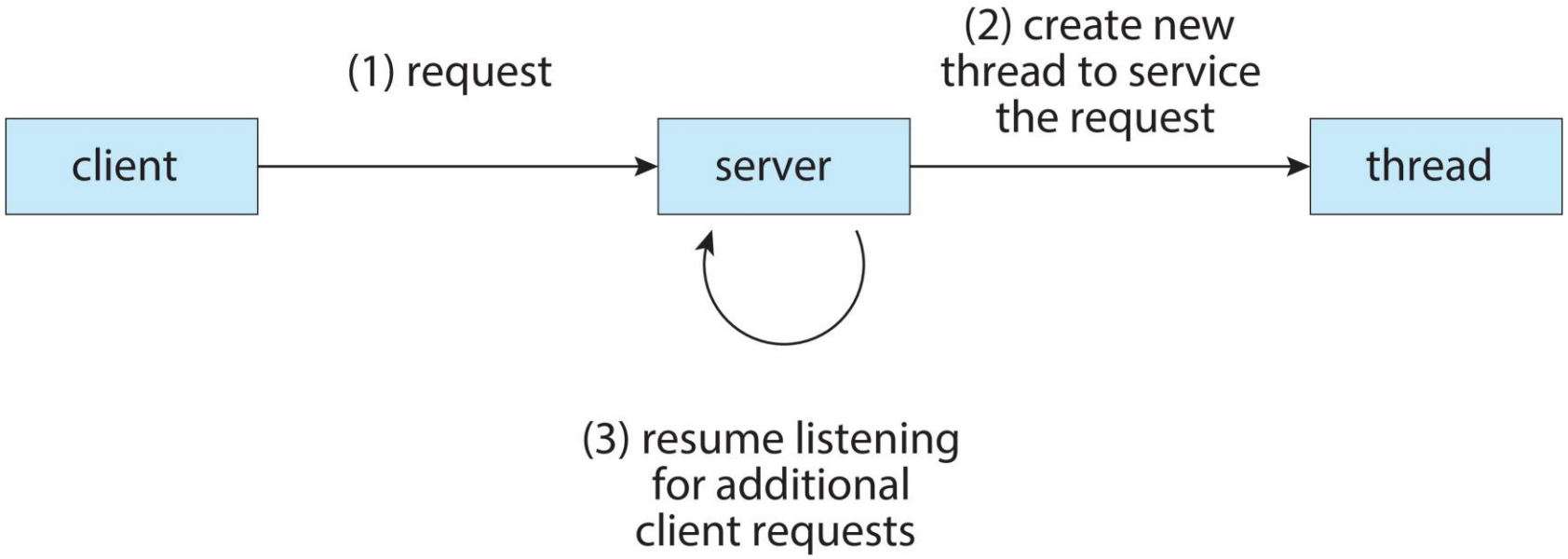
Threads

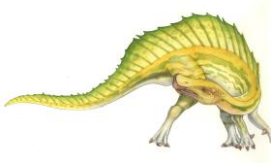

PCB
CPU_state
pc
sp
process_state
memory
TCB1
CPU_state
pc1
sp1
thread_state
TCB2
CPU_state
pc2
sp2
thread_state
code
data
stack 1
stack 2





Multithreaded Server Architecture

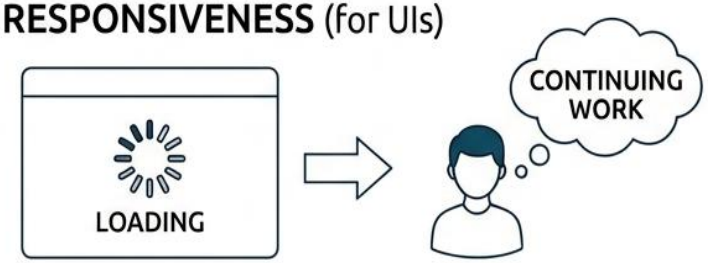




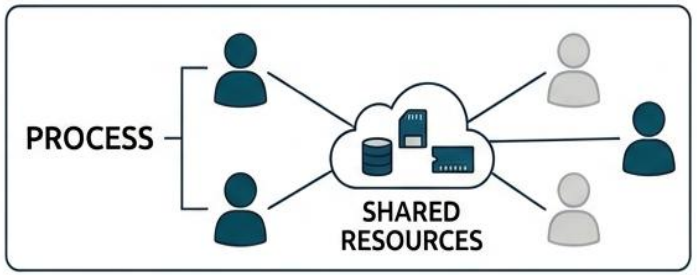


Benefits

1. **RESPONSIVENESS** (for UIs)



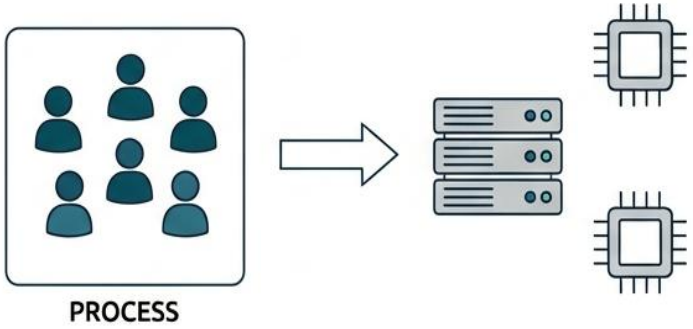
2. **RESOURCE SHARING** (within Process)

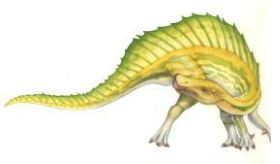


3. **ECONOMY** (Lower Cost & Overhead)



4. **SCALABILITY** (Efficient Growth)







Multicore Programming

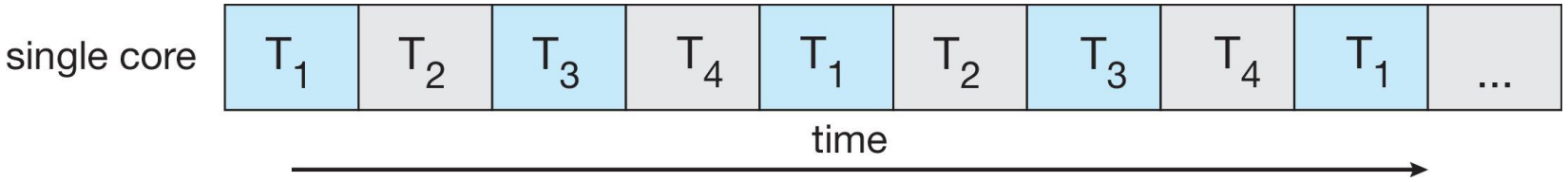
- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency



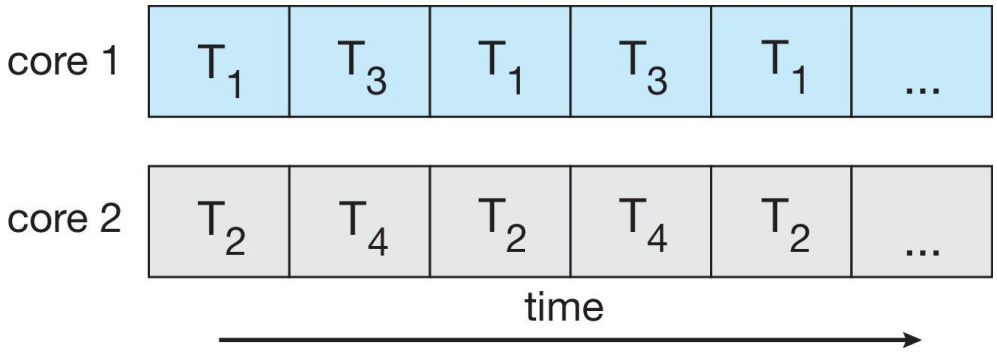


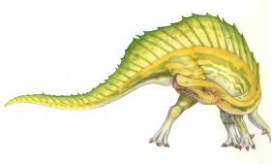
Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



- **Parallelism on a multi-core system:**

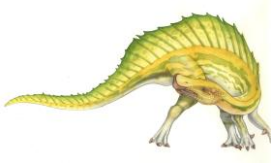






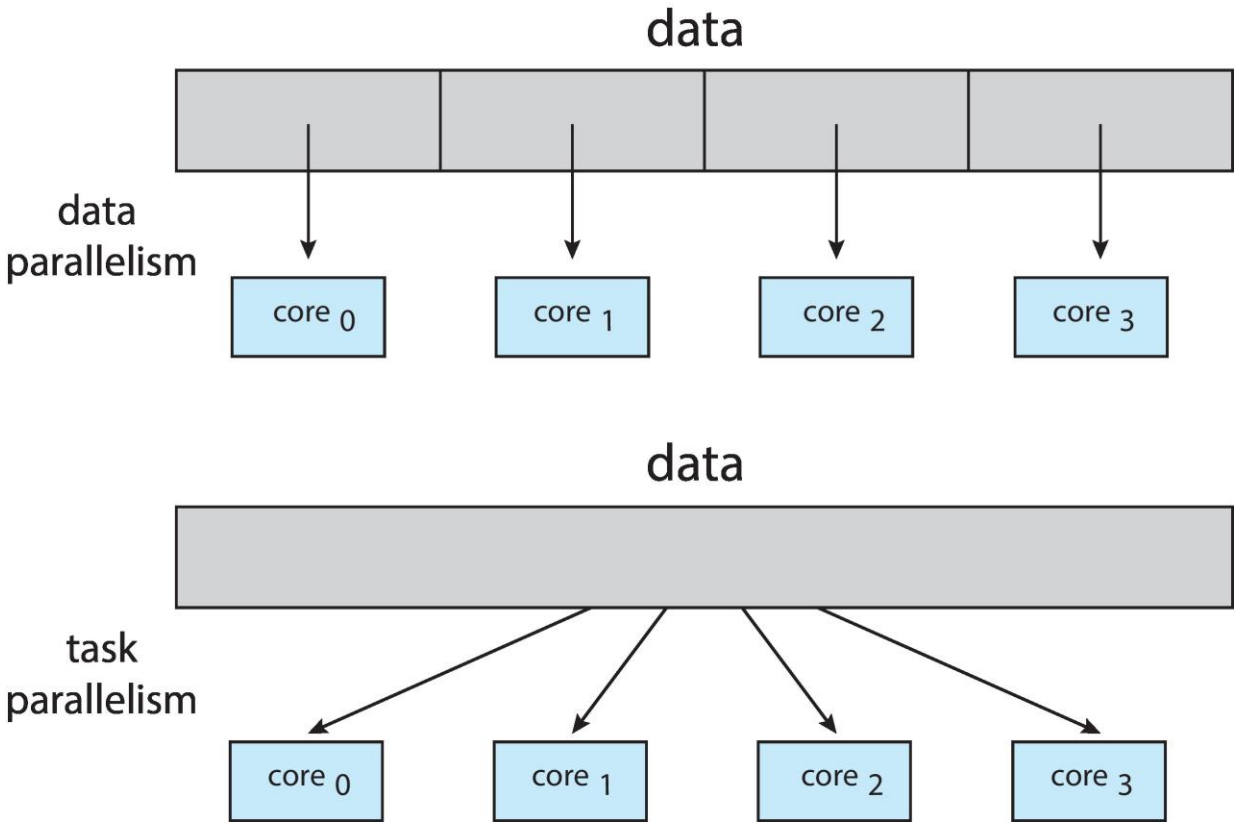
Multicore Programming

- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation





Data and Task Parallelism

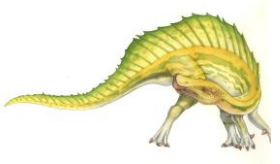






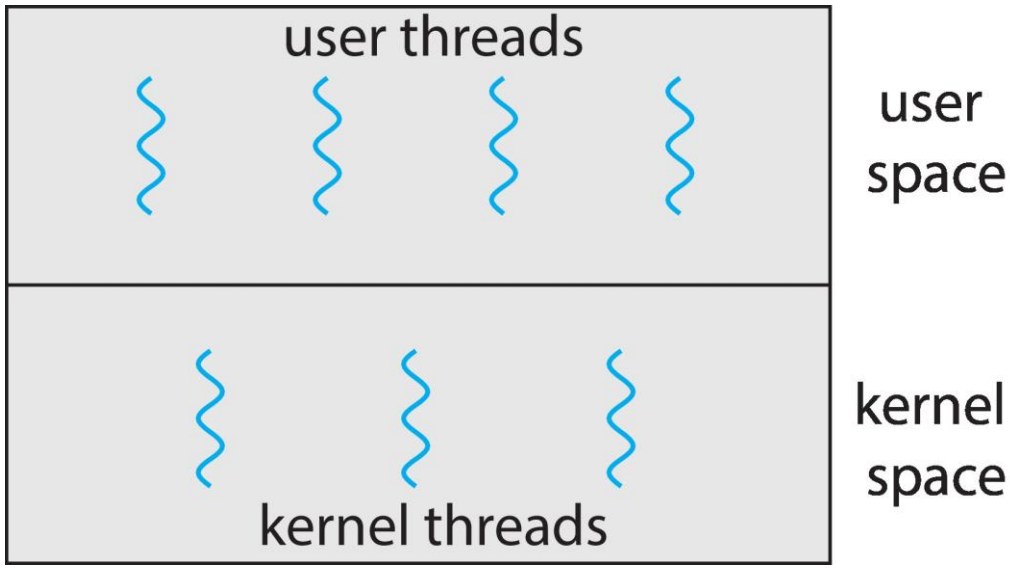
User Threads and Kernel Threads

- **User threads** - management done by user-level threads library
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general purpose operating systems, including:
  - Windows
  - Linux
  - Mac OS X
  - iOS
  - Android





User and Kernel Threads

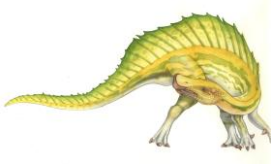






Multithreading Models

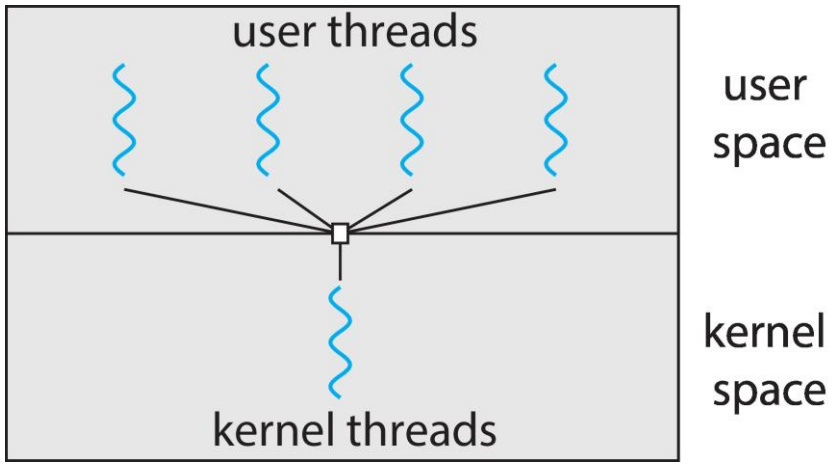
- Many-to-One
- One-to-One
- Many-to-Many





Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**

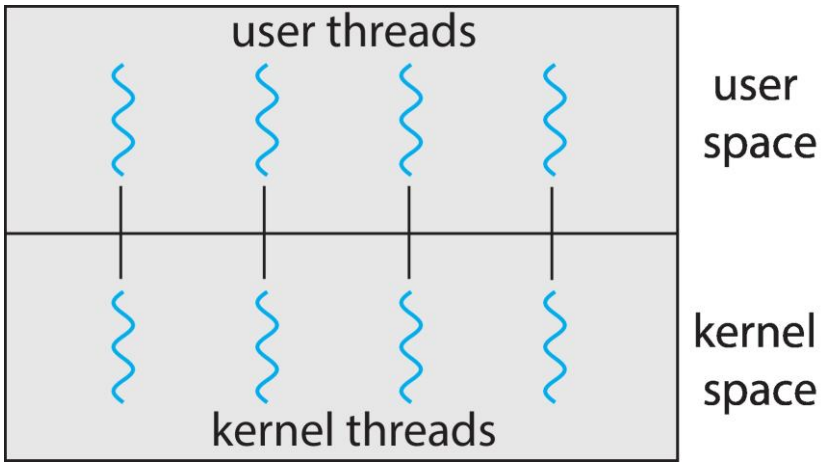






One-to-One

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux

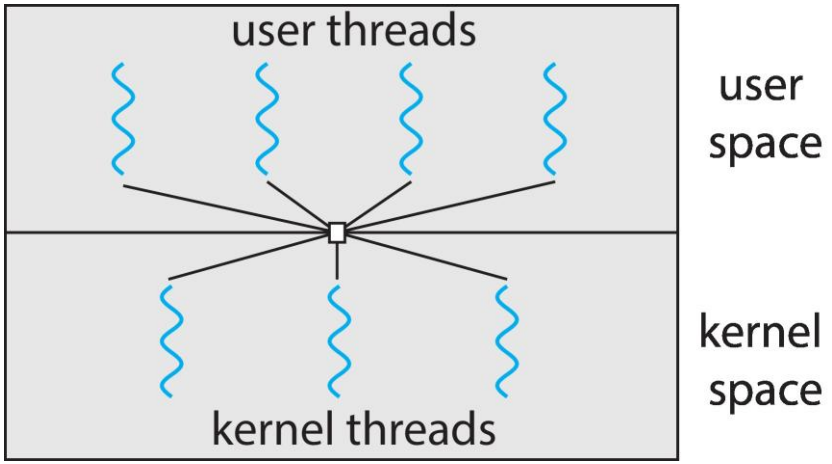


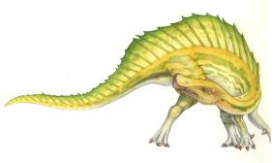




Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Windows with the *ThreadFiber* package
- Otherwise not very common

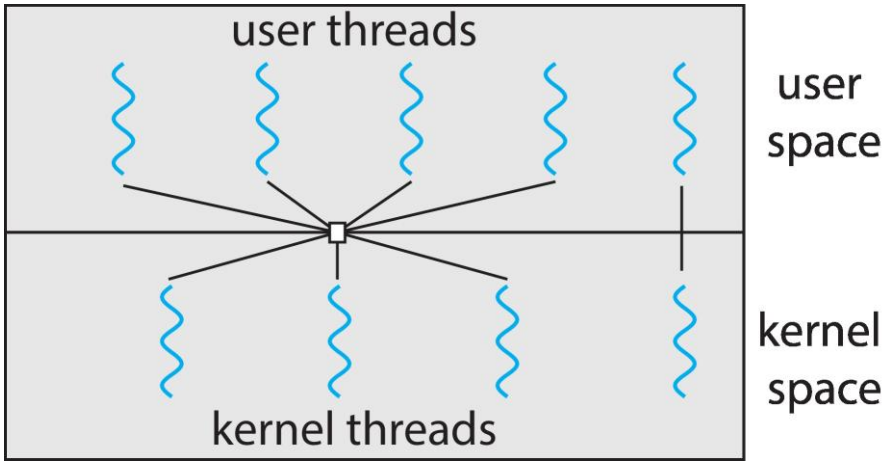


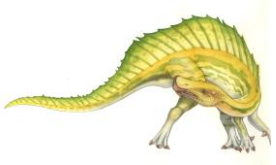




Two-level Model

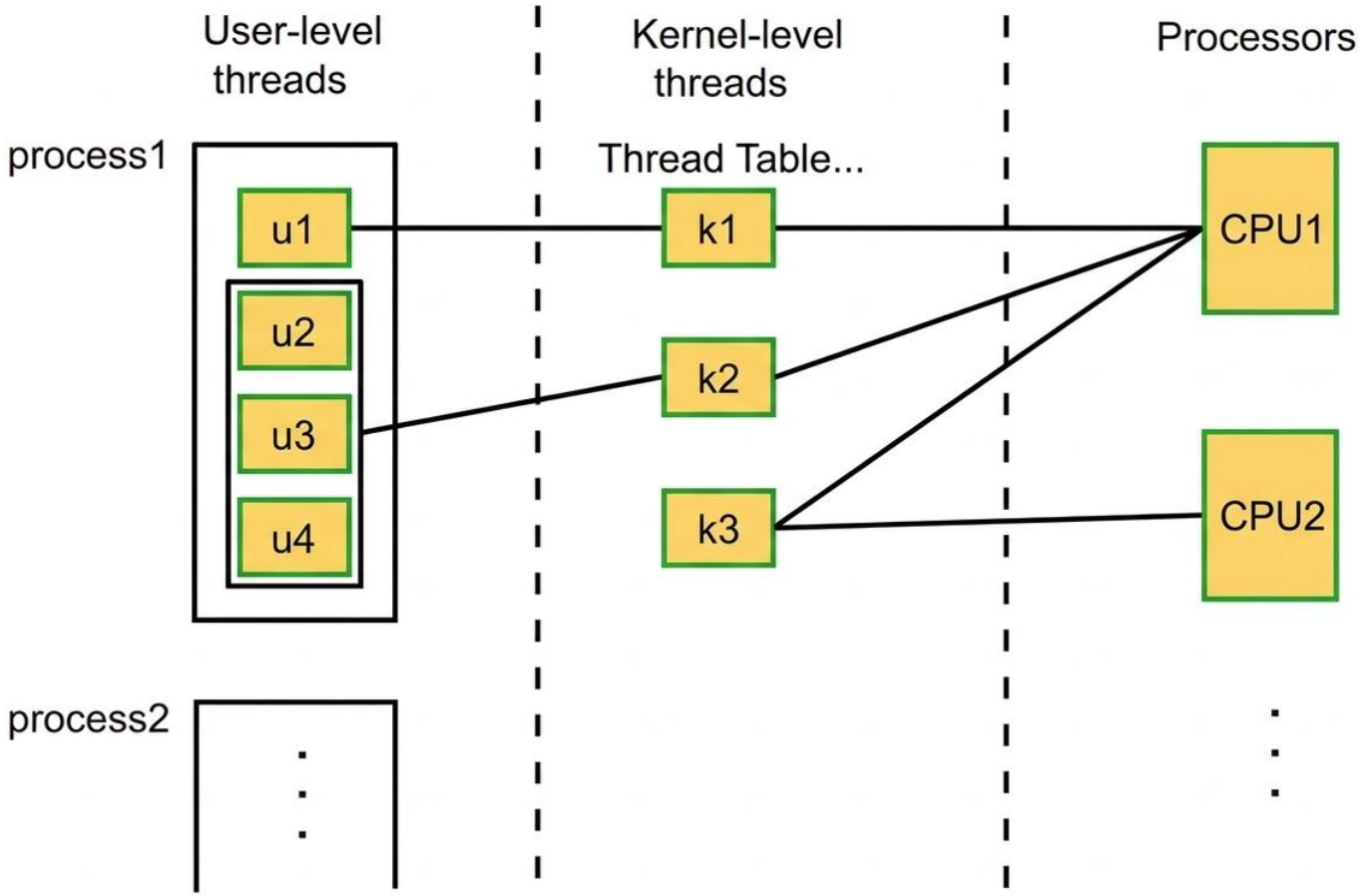
- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread







User and Kernel Threads







Example

- **Question**
  - A **web application server** receives and processes requests from users. The server keeps the results of frequently requested pages in memory as a cache.
  - A new request arrives on average every **40 ms**.
  - The processing of each request takes **20 ms**.
  - If the requested page is not found in the memory cache, an additional **100 ms** is required to retrieve the page from storage.
  - On average, **70%** of all requests can be serviced without accessing storage.
  - The creation of a new thread takes **5 ms**.
  - **(a)**
  - Should the server be implemented as a **single-threaded** or **multi-threaded** process? Justify your answer.
  - **(b)**
  - What percentage of requests would have to be satisfied **without accessing storage** for the single-threaded approach to be feasible?





Example

Step 1: Calculate the processing time for each type of request.
•70% of requests take **20 ms**.
•30% of requests require storage access: 20+100=120 ms

•For a **single-threaded** process, the average time per request is:
0.7(20)+0.3(120) =14+36=50 ms

A new request arrives every **40 ms**, but the server requires an average of **50 ms** to process each request.
Since: 50>40
the single-threaded process **cannot keep up with incoming requests**. An unbounded queue will eventually form.

For a **multi-threaded** process, the thread can perform other work while a request is waiting for storage access.
The average CPU processing time is: 5+20=25 ms
where:
•5 ms = thread creation
•20 ms = request processing
Since: 25<40
the multi-threaded server **can keep up with incoming requests**.





Example

Let n be the fraction of requests that can be serviced **without storage access**. Then 1−n is the fraction requiring storage access.

For the single-threaded approach to be feasible, the average processing time must be **no more than 40 ms**:

$n(20)+(1-n)(120) \leq 40$

$\Rightarrow 20n+120-120n \leq 40$

$\Rightarrow 120-100n \leq 40$

$\Rightarrow -100n \leq -80$

$\Rightarrow n \geq 0.80$

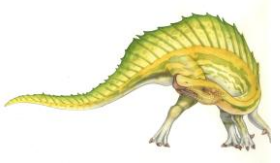
At least **80% of requests must be satisfied without accessing storage** for the single-threaded approach to be feasible.





Thread Libraries

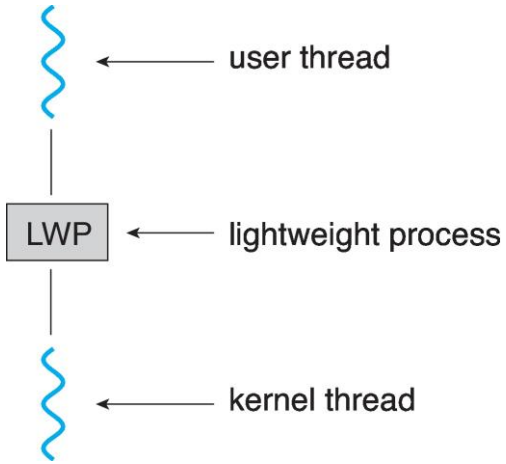
- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
  - Library entirely in user space
  - Kernel-level library supported by the OS





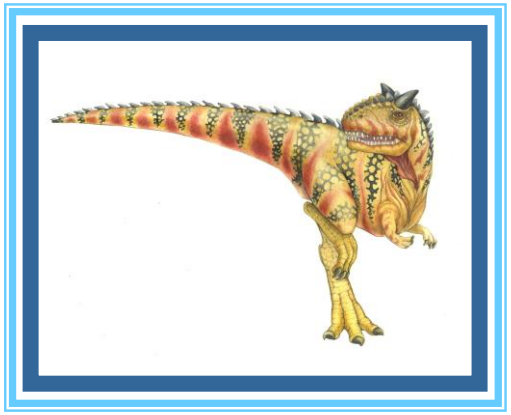
Scheduler Activations

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application
- Typically use an intermediate data structure between user and kernel threads – **lightweight process** (**LWP**)
  - Appears to be a virtual processor on which process can schedule user thread to run
  - Each LWP attached to kernel thread
  - How many LWPs to create?
- Scheduler activations provide **upcalls** - a communication mechanism from the kernel to the **upcall handler** in the thread library
- This communication allows an application to maintain the correct number kernel threads





End of Chapter 4



Silberschatz, Galvin and Gagne ©2018